

01. TITLE SLIDE

~1 min

CONTENT

Session 12: Real-World Case Studies & Capstone

Welcome to Session 12—the final session. You've completed 11 sessions. You understand LLMs, you can prompt them, structure outputs, ground them, specialize them, evaluate them, scale them, optimize them, make them safe, and design cutting-edge architectures. Today: See how real companies built real systems. Learn lessons from production. Then: Build your own complete end-to-end AI system. This is where theory meets practice. This is where you become an AI engineer.

02. TODAY'S JOURNEY

~2 min

CONTENT

Case studies, lessons, complete system design, capstone, career

Quick roadmap. Three minutes on why case studies matter. Eight minutes each on five real-world systems: Netflix recommendation, Google Search, OpenAI GPT, medical AI, autonomous driving. Five minutes on lessons learned. Five minutes on complete system design framework. Three minutes on capstone. Five on AI career. The capstone is your chance to design a complete system from scratch using everything from Sessions 1-11.

03. WHY CASE STUDIES MATTER

~3 min

CONTENT

Learn from real production systems

Case studies teach: What works, what doesn't, why. Real constraints. Real tradeoffs. Real consequences. Netflix: 250M users, recommendations matter. How did they solve it? What architecture? Google Search: 8.5B searches/day, sub-100ms latency. How? OpenAI GPT: State-of-the-art language models. How built? Medical AI: Life-or-death accuracy requirements. How tested? Autonomous driving: Safety critical. How deployed? Each case teaches different lessons. Together: Picture of how real AI systems work. Where theory meets reality.

04. CASE STUDY 1: NETFLIX RECOMMENDATION

~8 min

CONTENT

250M users, personalization, engagement, real-time

Problem: 10,000 titles, 250M users. Which ones to recommend to each user? Traditional: Show most popular. Problem: Boring, doesn't personalize. Netflix solution: (1) Collaborative filtering: Users who watched X also watched Y. If you watched X, recommend Y. (2) Content-based: Recommend similar to what you watched. (3) Matrix factorization: Users and items in latent space. Similar items = similar vectors. (4) Deep learning: Neural networks learn user-item interactions. (5) Multi-armed bandits: Explore sometimes (new things), exploit usually (known good). (6) Context: Time of day, device, location affect recommendations. Architecture evolution: (1) Year 1: Simple collaborative filtering. (2) Year 3: Added deep learning. (3) Year 5: Added bandits for exploration. (4) Year 7: Real-time personalization. (5) Today: Mixture of techniques, context-aware, personalized. Impact: 5-10% increase in engagement = 10M+ hours watched more per day = hundreds of millions in revenue. Lessons: (1) Start simple. (2) Iterate continuously. (3) Measure impact carefully. (4) Combine techniques. (5) Context matters.

05. CASE STUDY 2: GOOGLE SEARCH

~8 min

CONTENT

8.5B searches/day, sub-100ms latency, ranking

Problem: Query "machine learning". 100 billion relevant pages. Rank them by relevance. 8.5 billion queries per day. Return results in <100ms. How? Architecture: (1) Crawl: Billions of pages indexed. (2) Indexing: Inverted index (word→pages). (3) Retrieval: Given query, find relevant pages (maybe millions). (4) Ranking: Rank millions by relevance (ML models). (5) Serving: Return top results instantly. Ranking model: Hundreds of signals. Is query in title? (strong signal). Is query in URL? (strong). Page authority (PageRank). Click-through rate (users clicked this result for this query). Dwell time (users stayed on page). (And hundreds more). Neural network takes all signals, outputs relevance score. Latency challenge: Ranking billions takes time. Solution: (1) Multi-stage: Quick filter (1M pages). Slow rank (1K pages). Return top 10. (2) Pre-computation: Pre-rank popular queries. (3) Caching: Cache results. (4) Distributed: Parallel search across thousands of machines. Impact: If search latency increases 100ms, traffic decreases 1-2%. At Google's scale, that's millions of searches lost. So latency matters obsessively. Lessons: (1) Latency is feature. (2) Multi-stage systems. (3) Pre-computation and caching critical. (4) Distributed systems necessary. (5) Measure everything.

06. CASE STUDY 3: OPENAI GPT DEVELOPMENT

~8 min

CONTENT

Foundation models, scaling, training, deployment

Problem: Language understanding and generation. Build system that understands and generates human language. OpenAI approach: Scaling laws. More data + more parameters + more compute = better models. GPT-1 (2018): 117M parameters. GPT-2 (2019): 1.5B parameters. GPT-3 (2020): 175B parameters. GPT-4 (2023): ~1.7T parameters (with MoE). Training: GPT-3 trained on 300B tokens (Wikipedia, books, web). Took months on TPU clusters. Cost: Tens of millions. Approach: (1) Unsupervised pre-training: Predict next token given previous tokens. (2) Few-shot learning: Give examples, model learns. (3) Instruction following: Fine-tune to follow instructions. (4) RLHF (Reinforcement Learning from Human Feedback): Humans rate responses, model learns to produce better responses. Deployment: (1) API: Companies use via API. (2) Cost: Per-token pricing. (3) Safety: Content filtering, rate limiting. (4) Monitoring: Usage patterns, abuse detection. Impact: GPT-3 can write essays, code, explain concepts. GPT-4: Better accuracy, multi-modal, more reliable. Impact on industry: Foundation models. Every AI system now builds on these. Lessons: (1) Scaling works. (2) Pre-training then fine-tuning. (3) RLHF improves alignment. (4) API deployment. (5) Continuous improvement.

07. CASE STUDY 4: MEDICAL AI FOR DIAGNOSIS

~8 min

CONTENT

High stakes, accuracy critical, regulatory

Problem: Doctors diagnose diseases. Millions of patients. Shortages of specialists. Can AI help? Medical AI systems: (1) Radiology: Detect cancer in X-rays, CT scans. (2) Pathology: Identify tumors in tissue samples. (3) ECG: Detect heart conditions. Challenges: (1) Accuracy critical. Mistake = patient harm or death. (2) Data private. HIPAA regulations. (3) Datasets small. Medical data hard to get. (4) Bias. Models trained on predominantly one demographic bias against others. Real system (Chest X-ray diagnosis): (1) Data: 15,000 X-ray images with diagnoses. (2) Model: Deep CNN (ResNet-50). (3) Performance: 95% accuracy on test set. (4) Reality: Deployed, found 87% accuracy in real hospital (domain shift). (5) Problem: Training data from one hospital, deployment in different hospital with different equipment, different patient population. (6) Solution: Continuous monitoring, retraining on new data. (7) Safety: Model never makes final decision. Alerts radiologist. Radiologist makes decision. (8) Fairness: Tested on men/women, different ages, different races. Required equal performance. Lessons: (1) High stakes = high scrutiny. (2) Domain shift is real. (3) Fairness testing mandatory. (4) Never trust model alone. (5) Continuous monitoring post-deployment. (6) Regulatory compliance critical.

08. CASE STUDY 5: AUTONOMOUS DRIVING

~8 min

CONTENT

Safety critical, real-time, edge computing

Problem: Drive a car without human. Millions of possible scenarios. Need to handle all safely. Approach: (1) Perception: Cameras, lidar, radar detect surroundings. (2) Prediction: Predict what other vehicles will do. (3) Planning: Plan safe trajectory. (4) Control: Execute trajectory. Architecture: (1) On-device inference. Can't use cloud (latency too high, no connectivity). (2) Edge GPUs. Process video streams. (3) Multiple models. Redundancy for safety. (4) Sensor fusion. Combine camera, lidar, radar. Challenges: (1) Edge cases. What if you see something training data didn't cover? (2) Adversarial. Adversaries deliberately create edge cases (parking signs modified). (3) Failure modes. What if GPU fails? What if prediction wrong? (4) Safety critical. One mistake = crash = death. Tesla approach: (1) Data collection. 4M cars collecting video. Billions of miles. (2) Continuous learning. Find failures, label them, retrain. (3) Multiple models voting. If they disagree, don't execute. (4) Human fallback. Driver can take over. Deployment: (1) Beta testing. Limited users, monitored closely. (2) Gradual rollout. More users as confidence increases. (3) Continuous monitoring. Fleet monitoring for failures. Lessons: (1) Safety-critical = conservative. (2) Edge cases everywhere at scale. (3) Redundancy and fallback essential. (4) Continuous learning from failures. (5) Slow, careful deployment.

09. LESSONS LEARNED ACROSS ALL CASES

~5 min

CONTENT

Common patterns, what works, what doesn't

Pattern 1: Start simple, iterate. Netflix didn't start with deep learning. Google didn't start with billions of pages. Build minimum viable system, improve. Pattern 2: Measure everything. Netflix: Engagement. Google: Latency, relevance. Medical: Accuracy by demographic. Autonomous: Safety metrics. If you don't measure, you don't know if you're improving. Pattern 3: Real-world is different from lab. Medical AI: 87% real hospital vs 95% test. Autonomous: Edge cases appear only at scale. Deploy carefully, monitor closely. Pattern 4: Fairness and safety critical. Medical: Different performance for different races = liability. Autonomous: One mistake = death. Safety testing mandatory. Pattern 5: Data is everything. Netflix: 250M users = training signal. Google: Billions of pages = ranking signal. Medical: Small dataset = poor performance. OpenAI: 300B tokens = powerful model. Collect more diverse data. Pattern 6: Combine techniques. Netflix: Collaborative + content-based + bandits + context. Single technique usually not enough. Combination often wins. Pattern 7: People matter. Netflix team understood recommendation. Google team understood search. Medical team understood medicine. Hiring smart people. Pattern 8: Long-term thinking. Netflix invested years. Google invested decades. OpenAI invested billions. Quick fixes don't work. Commit to long-term improvement.

10. COMPLETE SYSTEM DESIGN FRAMEWORK

~5 min

CONTENT

How to design end-to-end system

Step 1: Understand problem. What's the goal? Netflix: Engagement. Google: Relevance + latency. Medical: Accuracy. Autonomous: Safety. Be specific. Step 2: Define constraints. Budget? Latency? Privacy? Scale? Netflix: 250M users, must handle real-time. Google: 8.5B searches, <100ms. Medical: High accuracy, fairness. Step 3: Choose architecture. Based on constraints. Netflix: Neural networks, bandits. Google: Multi-stage ranking. Medical: CNN. Autonomous: Real-time CNN. Step 4: Collect data. What data do you need? How much? Is it representative? Netflix: User interactions. Google: User clicks. Medical: Labeled images. Autonomous: Video, lidar, radar. Step 5: Build baseline. Simplest system that works. Netflix: Popular items. Google: Keyword matching. Medical: Rule-based. Autonomous: Rule-based. Step 6: Improve incrementally. Add complexity only if justified. Netflix: Add ML if better than baseline. Google: Add ranking model if faster and more relevant. Step 7: Test thoroughly. Lab testing. Staging testing. Limited deployment. Full deployment. Medical, Autonomous: Even more testing. Step 8: Monitor continuously. Measure metrics post-deployment. Netflix: Engagement trends. Google: Relevance, latency. Medical: Accuracy by demographic. Autonomous: Safety metrics. Step 9: Iterate. Find failures, learn, improve. System never finished. Always improving.

11. CAPSTONE PROJECT OVERVIEW

~3 min

CONTENT

Design complete end-to-end system

Your task: You now have a hiring AI system (Sessions 1-11). Design complete end-to-end production system. What does "production-ready" mean? (1) Works at scale. (2) Handles edge cases. (3) Fair and safe. (4) Monitorable. (5) Optimized. (6) Easy to improve. Capstone has 8 parts: (1) Problem definition (what's goal?). (2) Constraints (budget, latency, scale?). (3) Architecture design (using Sessions 11 knowledge). (4) Data strategy (what data, how much?). (5) Evaluation plan (how to test?). (6) Deployment strategy (safe rollout). (7) Monitoring plan (what to measure?). (8) Scaling plan (what if 10x traffic?). You'll design COMPLETE system from Sessions 1-11.

12. YOUR AI ENGINEERING CAREER

~5 min

CONTENT

Paths forward, next skills, opportunities

Career paths: (1) AI research: Universities, labs. Push boundaries. Focus on new techniques. (2) AI product: Companies. Build systems users use. Focus on impact. (3) AI engineer: Companies. Build and deploy systems. Focus on reliability. (4) AI safety: Companies, organizations. Focus on safety and alignment. (5) Entrepreneurship: Start company. Build own product. Skills to develop next: (1) Systems design: How to build large-scale systems. (2) Deployment: Getting models to production. (3) DevOps: Monitoring, reliability. (4) Domain expertise: Deep knowledge in one domain. (5) Leadership: Managing teams. Opportunities: (1) Exploding field. Every company needs AI now. (2) Massive funding. VCs investing billions. (3) Shortage of talent. Supply < demand. (4) Impact. AI changing world. (5) Compensation. AI engineers highly paid. Next steps: (1) Keep learning. AI evolving fast. (2) Build projects. Portfolio matters. (3) Contribute to open source. Visibility. (4) Network. Community matters. (5) Specialize. Pick domain, become expert.

13. KEY TAKEAWAYS FROM COMPLETE COURSE

~5 min

CONTENT

Wisdom from Sessions 1-12

Takeaway One: AI is not magic. It's mathematics, data, and engineering. Understand fundamentals. Takeaway Two: Real systems are messy. Theory is clean. Production is constraints, tradeoffs, iteration. Takeaway Three: Data is king. Good data > good algorithm (usually). Collect, clean, curate carefully. Takeaway Four: Simple often wins. Complex solutions look impressive. Simple solutions work. Start simple. Takeaway Five: Fairness and safety are non-negotiable. Not afterthought. Build in from day one. Takeaway Six: Scaling is hard. Different problems at different scales. Monitor continuously. Takeaway Seven: Iteration is key. Perfect systems don't exist. Better systems do. Keep improving. Takeaway Eight: People matter. Hire smart people. Collaborate. Diversity builds better systems.

14. CLOSING REMARKS

~2 min

CONTENT

You're now an AI engineer

You've completed 12 sessions. You understand AI deeply. You can build, deploy, optimize, and make systems safe. You're an AI engineer. The journey doesn't end here. AI is evolving rapidly. Stay curious. Keep learning. The future is being built by people like you. Build responsibly. Thank you for this journey. Go build amazing things.